AI-TECH

Przykład 4 DO LABORATORIUM 2

z UCZENIA MASZYNOWEGO

Klasyfikacja nadzorowana

Szymon Zaporowski

Porównanie klasyfikatorów na przykładzie danych syntetycznych

Projekt współfinansowany ze środków Unii Europejskiej w ramach Europejskiego Funduszu Rozwoju Regionalnego Program Operacyjny Polska Cyfrowa na lata 2014-2020, Oś Priorytetowa nr 3 "Cyfrowe kompetencje społeczeństwa" Działanie nr 3.2 "Innowacyjne rozwiązania na rzecz aktywizacji cyfrowej" Tytuł projektu: „Akademia Innowacyjnych Zastosowań Technologii Cyfrowych (AI Tech)”

Przykład 4 : Porównanie klasyfikatorów

Ten przykład przedstawia porównanie kilku klasyfikatorów zaimplementowanych w ramach pakietu scikit-learn z wykorzystaniem syntetycznych zbiorów danych. Przykład ilustruje naturę granic decyzyjnych różnych klasyfikatorów. Należy mieć na względzie fakt, iż są to generowane sztucznie zbiory danych i taki układ może niekoniecznie przenosić się na rzeczywiste dane. To przedstawienie granic służy do pokazania procesu klasyfikacji dla różnych klasyfikatorów i porównania sposobu ich działania na tych samych danych.

Wykresy przedstawiają punkty treningowe z wykorzystaniem jednolitych kolorów, natomiast punkty testowe są reprezentowane jako półprzezroczyste. W prawym dolnym rogu przedstawiona została miara dokładności klasyfikacji z użyciem zbioru testowego.

In [5]:
```
#Import niezbędnych bibliotek

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.colors import ListedColormap
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.datasets import make_moons, make_circles, make_classification
from sklearn.neural_network import MLPClassifier
from sklearn.neighbors import KNeighborsClassifier
```

```python
from sklearn.svm import SVC
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import RBF
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
%matplotlib inline
```

In [6]:

```python
# Inicjalizacja klasyfikatorów oraz syntezowania danych


names = ["Regresja Logistyczna", "SVM z jądrem liniowym", "SVM z jądrem RBF","SVM z jądrem wielomianowym",
         "Drzewo Decyzyjne", "Klasyfikator Random Forest", "Regresja Logistyczna","OVR"]

classifiers = [
    LogisticRegression(solver='newton-cg',random_state=0), #Regresja
    SVC(kernel="linear", C=0.025), #SVM
    SVC(gamma=2, C=100), #SVM
    SVC(kernel="poly",degree=3, gamma=2, C=1), #SVM
    DecisionTreeClassifier(max_depth=5),
    RandomForestClassifier(max_depth=5, n_estimators=10, max_features=1),
    ]

#tworzenie losowych danych
X, y = make_classification(n_features=2, n_redundant=0, n_informative=2,
                           random_state=0, n_clusters_per_class=1)
rng = np.random.RandomState(3)
X += 2 * rng.uniform(size=X.shape)
linearly_separable = (X, y)

datasets = [make_moons(noise=0.3, random_state=0),
            make_circles(noise=0.2, factor=0.5, random_state=1),
            linearly_separable
            ]
print("Dataset:", datasets)
```

```
Dataset: [(array([[ 0.03159499,  0.98698776],
       [ 2.11509784, -0.04624397],
       [ 0.88248972, -0.07575606],
       [-0.0551441 , -0.03733246],
       [ 0.82954503, -0.53932149],
       [ 2.11285708,  0.66208353],
       [ 0.5696927 ,  0.33744136],
       [ 0.95217454, -0.75307471],
       [-0.02922115,  0.39287336],
       [ 1.71579557,  0.36069454],
       [-0.27883014,  0.61420539],
       [ 2.00344509, -0.3091496 ],
       [ 1.50009616, -0.74636376],
       [ 1.22540835,  1.19793017],
       [ 0.46730819,  0.54703192],
       [-0.36653222,  1.11971633],
       [ 0.30124459,  1.23315697],
       [ 0.30430746,  0.82373935],
       [-0.23712492,  0.77876034],
       [ 1.16339832,  0.55290238],
       [-0.59702417,  0.6572361 ],
       [ 0.46356735, -0.1951543 ],
       [ 0.68451111,  0.75287685],
       [ 0.70515699, -0.45892444],
       [-0.65805008, -0.12944211],
       [-0.74662946, -0.3829632 ],
       [ 0.60585226,  0.31252842],
       [ 2.18137168, -0.02291747],
       [ 1.91980633,  0.17247329],
       [ 1.4834364 , -0.10517023],
       [ 0.47404723, -0.14575067],
       [ 1.05614605, -1.03570207],
       [ 1.86500732, -0.01107874],
       [ 0.4194721 , -0.32478101],
       [ 0.06873258,  0.56648467],
       [-0.17332432,  1.00215131],
       [ 0.12412421,  1.00062892],
       [ 0.18121142,  0.24717743],
       [-0.25451559,  0.19317272],
       [ 1.02580668, -0.62574566],
       [ 1.52002143, -0.04515581],
       [ 0.64174037, -0.39369468],
       [-0.87615589,  0.0465662 ],
       [-1.06964997,  0.13666025],
       [ 1.02658765,  0.37782458],
       [ 0.93131325,  1.38517841],
```

```
       [ 0.67680544,  1.57189931],
       [-0.36885733,  0.72886601],
       [-1.02465495,  0.16190215],
       [ 0.77455385,  0.15375803],
       [ 1.4045116 , -0.00705701],
       [-0.38147174, -0.23488747],
       [ 0.72155224,  0.44721658],
       [-0.51346686,  0.67869095],
       [ 0.32118546,  0.28668667],
       [ 0.1953628 ,  0.16085107],
       [ 0.52824196,  0.98300993],
       [-0.2216539 ,  0.25160139],
       [ 0.22334676,  1.32217183],
       [-0.10704572,  0.56178326],
       [ 0.63651685,  0.75444825],
       [-0.37227848,  0.99291317],
       [ 0.20718083, -0.09767143],
       [ 0.12733142, -0.3796549 ],
       [ 0.71435231, -0.79994161],
       [ 1.09487814, -0.36841845],
       [-0.14814362,  0.96158657],
       [ 1.586188  , -0.62984517],
       [ 0.74444551,  0.57661371],
       [ 2.18011028, -0.69977751],
       [ 0.24575594,  0.8496383 ],
       [-0.95003581,  0.90361699],
       [-0.88230758,  0.07249044],
       [ 2.02297079,  0.12325148],
       [ 2.14577321,  0.46296362],
       [ 0.35536468, -0.67847989],
       [ 0.34665026,  1.11570676],
       [ 1.7392373 ,  0.45900352],
       [ 0.63856467, -0.44718443],
       [ 1.2876687 , -0.4910366 ],
       [-0.13772607,  1.2453262 ],
       [-0.56175303,  1.05486051],
       [ 1.29003748, -0.20691405],
       [-0.87539365,  0.50543423],
       [-0.92858249, -0.45631991],
       [ 0.02493632,  0.10747958],
       [ 0.1972559 , -0.06801668],
       [ 0.73346056,  0.28161929],
       [ 1.68294434, -0.2020423 ],
       [ 0.50764124, -0.11731979],
       [ 1.66760217, -0.42485665],
       [-0.82172282,  0.63141066],
```

```
       [ 0.30170903,  0.78603534],
       [ 1.37671505, -0.80915107],
       [ 1.17037551,  0.59840653],
       [ 1.69945309,  0.58771967],
       [ 0.21862323, -0.65252119],
       [ 0.95291428, -0.41976564],
       [-1.31850034,  0.42311235],
       [-1.29681764,  0.18414709]]), array([0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1,
       0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1,
       0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 0,
       1, 1, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0])), (array([[-0.38289117, -0.09084004],
       [-0.02096229, -0.47787416],
       [-0.39611596, -1.28942694],
       [-0.61813018, -0.06383715],
       [ 0.70347838, -0.18703837],
       [-0.45970463,  0.69477465],
       [-0.45091682, -0.71570524],
       [-0.45562004, -0.13406016],
       [-0.26513904,  0.40812871],
       [-0.15474648,  0.41406973],
       [ 0.231206  , -0.53275899],
       [ 0.15623875, -0.8678088 ],
       [ 0.51647541,  0.48940995],
       [ 0.68707007, -0.02334129],
       [ 0.54759869, -0.16482373],
       [-0.32179572, -0.80553536],
       [-1.07478639, -0.733362  ],
       [ 0.76758455, -0.43498783],
       [-0.47475234, -0.33813186],
       [ 0.15420656,  1.07306032],
       [ 0.65916696,  0.20773634],
       [-0.77795003,  0.1326555 ],
       [ 0.47025454, -0.31228748],
       [-0.04246799,  0.26555446],
       [-0.72405954,  0.48807185],
       [-0.36960005, -1.06514028],
       [ 0.17833327, -0.49718972],
       [-0.93927864, -0.41951638],
       [ 0.50914152, -0.70977467],
       [-0.05569852, -0.82162607],
       [-0.11214579,  0.72197044],
       [ 0.80463921, -0.15221296],
       [ 0.08261487, -0.11749021],
       [ 0.20349541, -0.37396789],
       [ 0.13864693, -0.23905642],
```

```
       [ 0.32785307, -1.00769037],
       [ 0.88944061, -0.39117628],
       [-0.05837947,  0.28487039],
       [-1.0673653 ,  0.2204006 ],
       [-0.60071345, -0.69545189],
       [-0.03972324, -0.40936056],
       [ 0.39742085,  0.20621162],
       [-0.36941154,  0.0129811 ],
       [ 0.03573703,  0.46666229],
       [-0.56814999, -0.41288419],
       [ 0.41047299, -0.73640868],
       [ 0.88249707, -0.69004404],
       [ 0.06579822, -0.50458395],
       [-0.75737223, -0.0724028 ],
       [ 0.18316966,  0.08722007],
       [ 0.67248314, -0.41892665],
       [ 0.25898723,  0.39688645],
       [-1.1312983 ,  0.4810614 ],
       [ 1.0592844 ,  0.64490287],
       [ 0.41019663,  0.38790198],
       [ 0.95142029, -0.04089983],
       [-0.60492988,  0.43950906],
       [ 0.23314762, -0.81785711],
       [ 0.91067331,  0.30702075],
       [-0.45026472, -0.03724104],
       [-0.81396121, -0.64733959],
       [-0.23191338,  0.50533992],
       [-0.59760983,  0.28023168],
       [ 0.73960166, -0.84270281],
       [ 0.57294659, -0.31198928],
       [ 0.24821133, -0.54784509],
       [ 0.52127802,  0.94108005],
       [ 0.33973198,  0.10609978],
       [ 1.05339036, -0.02197593],
       [ 0.01327466, -0.63379502],
       [ 0.2422589 ,  0.49032064],
       [-0.89266612,  0.6345076 ],
       [ 0.1672566 ,  0.23548462],
       [-0.05611705,  0.38834099],
       [ 0.84695486,  0.81435811],
       [ 0.29976195, -0.07943031],
       [-0.1404762 ,  0.72486032],
       [-0.05482024,  0.18417328],
       [-0.24643884, -0.43283337],
       [-0.23460645,  0.6409442 ],
       [-1.13184893, -0.61964942],
```

```
       [-0.92413821, -0.45302089],
       [ 0.2225745 ,  0.77052597],
       [-0.69453765,  0.53014147],
       [-1.0362509 ,  0.77339965],
       [ 0.51880585,  0.30152232],
       [-0.77429541,  0.02553767],
       [ 0.71468326,  0.56869015],
       [-0.33875274,  0.46826063],
       [-0.34749244,  0.13441418],
       [ 1.12980796,  0.04281936],
       [-0.38308979,  0.79116812],
       [-0.07425141,  0.2184625 ],
       [-0.44945202, -0.05722266],
       [ 0.85783288,  0.63778888],
       [-0.47486203, -0.22498112],
       [ 0.12627243,  0.86978412],
       [-0.64736458, -0.36342437],
       [ 0.47440459,  1.01101585],
       [-0.38565772, -0.81031183]]), array([1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1,
       1, 1, 0, 0, 1, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0,
       1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 1, 0, 0, 1,
       0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0,
       1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0])), (array([[ 0.49618001,  2.71300373],
       [ 1.93670934,  0.97477773],
       [ 3.56626865,  2.89244403],
       [ 1.68778527,  1.22212639],
       [ 0.8246039 ,  2.04977982],
       [ 2.79365238, -1.2228421 ],
       [ 3.73128263, -0.93021087],
       [ 3.38834035,  2.20851339],
       [ 1.31983083, -0.38743329],
       [ 1.44845521,  1.93583652],
       [ 0.49473336,  2.46589258],
       [-0.39837746,  0.1628375 ],
       [ 1.4627342 ,  2.57118024],
       [ 0.18996548,  1.31555566],
       [ 2.30411835,  0.09002908],
       [ 0.26257709,  1.46518644],
       [ 2.15684163,  2.79597553],
       [ 1.92880561, -0.59549601],
       [ 2.75679729,  2.34142954],
       [ 0.25781592,  1.30857485],
       [ 0.61184967,  0.84790908],
       [ 1.73218307,  1.75037691],
       [ 1.33971788,  2.53120517],
       [ 2.44031021,  1.29662586],
```

```
       [ 1.73956969,  1.83001644],
       [-0.05357728,  1.34171418],
       [ 1.00527202,  1.00229356],
       [-0.17113172,  0.77134697],
       [ 3.22072938,  0.84430735],
       [ 3.87075602, -1.12939907],
       [ 1.72585642,  0.68504682],
       [ 1.53677408,  1.94725724],
       [ 2.04122688, -0.31942011],
       [ 2.59890425, -1.14746655],
       [ 1.97077765, -1.01492897],
       [ 2.47575759, -0.81117926],
       [ 1.85484775, -0.26797504],
       [ 0.67173205, -0.59147751],
       [ 0.3638685 ,  1.14056811],
       [ 1.67364462,  1.14328747],
       [ 1.60262097, -0.58110197],
       [ 2.72924262,  2.88254785],
       [ 1.74138143,  0.03712322],
       [ 5.10558289, -2.29263211],
       [ 2.15876998,  1.40155411],
       [ 3.24709013,  2.05713596],
       [ 4.46801877,  2.36105125],
       [ 2.33451925,  1.16623234],
       [ 3.09973974,  1.68188485],
       [ 1.13538374,  0.12262291],
       [ 1.1827359 ,  2.21528802],
       [ 0.4599629 ,  3.27962964],
       [ 3.19215012, -0.66460462],
       [-0.11020651,  1.77400473],
       [ 5.16901415,  0.22198883],
       [ 2.92225784,  1.79153417],
       [ 2.433496  , -1.19417877],
       [ 1.67877654, -0.9258266 ],
       [ 2.2015259 ,  1.38202855],
       [ 2.47372497, -1.9799693 ],
       [ 2.39573571,  2.24726885],
       [ 2.63373677,  1.72684831],
       [ 1.78892313,  0.44994192],
       [ 2.23294848,  2.02102248],
       [ 0.47896449, -0.18464578],
       [ 1.0315675 ,  2.77085661],
       [ 2.96425564,  2.489204  ],
       [ 0.75724745,  1.45130726],
       [ 2.42241347,  1.02078434],
       [ 2.80557617,  1.28344994],
```

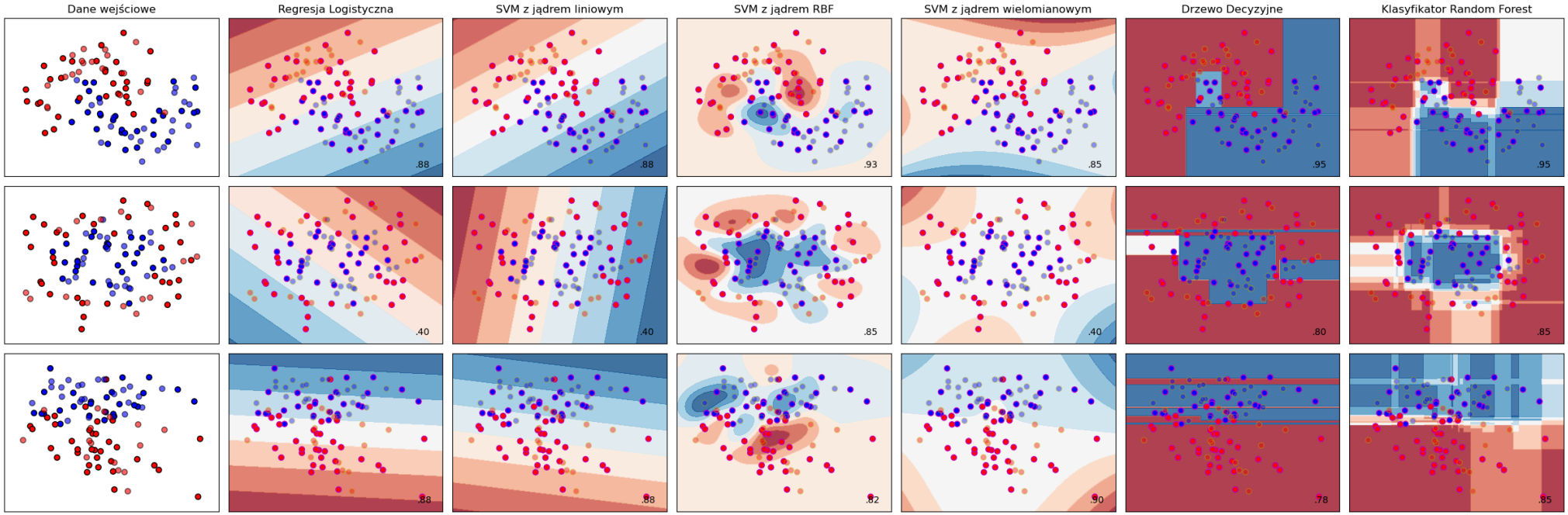
```
       [ 3.34983021,  1.58380185],
       [ 2.37429862, -0.75137548],
       [ 3.28506485, -0.06695849],
       [ 1.39921757,  1.82792468],
       [ 2.1625961 ,  1.03150349],
       [ 1.31950372, -0.23755304],
       [ 2.44208584,  1.17814934],
       [ 1.96522499,  2.79413389],
       [ 3.20986254,  1.61795261],
       [-0.10482477,  0.64763922],
       [ 4.04963168,  0.61197697],
       [ 0.26703793,  2.25476045],
       [ 1.89450152,  0.4317653 ],
       [ 1.94929557,  1.54399572],
       [ 1.61737252, -1.22879759],
       [ 2.89638674, -2.05374336],
       [ 1.68905224,  0.59753339],
       [ 1.76300836, -0.23726542],
       [ 0.80981615,  2.45417673],
       [ 1.56685914,  1.61640361],
       [ 3.97643016,  2.39710166],
       [ 0.59794403,  1.21639564],
       [ 2.21597585,  2.79990993],
       [ 1.68773751,  0.13879556],
       [ 1.19855455,  1.62278778],
       [ 2.46726994,  1.19679062],
       [ 2.46509235,  2.16706467],
       [ 2.55346264, -0.14282198],
       [ 0.77493438,  0.86595037],
       [ 4.98845438,  1.84430131]]), array([1, 0, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0,
       0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 0, 1, 1, 0, 1, 0, 1,
       1, 1, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       1, 0, 1, 0, 0, 0, 1, 1, 1, 0, 1, 1]))]
```

Zilustrujmy przykładowe porównanie klasyfikatorów typu SVM, Regresja Logistyczna oraz Drzew Decyzyjnych. Po lewej stronie wykresu znajdują się wygenerowane dane, poźnmiej pojawiają sie kolejne klasyfikatory. Dane treningowe zostały zaznaczone z wyraźną czerwoną obwódką, natmoast dane testowe są delikatnie przezroczyste.

In [7]:
```
figure = plt.figure(figsize=(24, 8))
i = 1
h = .01  # ustawienie rozmiaru siatki na wykresie
# iteracja po datasetach
for ds_cnt, ds in enumerate(datasets):
    # preprocesing danych, podział na dane testowe i treningowe
```

```
    X, y = ds
    X = StandardScaler().fit_transform(X)
    X_train, X_test, y_train, y_test = \
        train_test_split(X, y, test_size=.4, random_state=42)

    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    # Rysowanie zbioru danych
    cm = plt.cm.RdBu #YlGnBu#RdBu ##FF0000', '#0000FF'
    cm_bright = ListedColormap(['#FF0000', '#0000FF']) #00FF00', '#0000FF'])
    ax = plt.subplot(len(datasets), len(classifiers) + 1, i)

    if ds_cnt == 0:
        ax.set_title("Dane wejściowe")
    # Rysowanie punktów należacych do zbioru trengowego
    ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=cm_bright,
               edgecolors='k')
    # Rysowanie punktów należacych do zbioru testowego
    ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=cm_bright, alpha=0.6,
               edgecolors='k')
    ax.set_xlim(xx.min(), xx.max())
    ax.set_ylim(yy.min(), yy.max())
    ax.set_xticks(())
    ax.set_yticks(())
    i += 1

    # Iteracja po klasyfikatorach
    for name, clf in zip(names, classifiers):
        ax = plt.subplot(len(datasets), len(classifiers) + 1, i)
        clf.fit(X_train, y_train)
        score = clf.score(X_test, y_test)

        # Rysowanie granic decyzyjnych. W tym celu następuje przypisanie do każdego punktu na siatce [x_min, x_max]x[y_min, y_max].
        if hasattr(clf, "decision_function"):
            Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
        else:
            Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]

        # Wstawianie rezultatów klasyfikacji na wykres
        Z = Z.reshape(xx.shape)
        ax.contourf(xx, yy, Z, cmap=cm, alpha=.8)

        # Rysowanie punktów należacych do zbioru treningowego
```

```
        ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=cm_bright,
                   edgecolors='m')
        # Rysowanie punktów należacych do zbioru testowego
        ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=cm_bright,
                   edgecolors='y', alpha=0.4)

        ax.set_xlim(xx.min(), xx.max())
        ax.set_ylim(yy.min(), yy.max())
        ax.set_xticks(())
        ax.set_yticks(())
        if ds_cnt == 0:
            ax.set_title(name)
        ax.text(xx.max() - .3, yy.min() + .3, ('%.2f' % score).lstrip('0'),
                size=10, horizontalalignment='right')
        i += 1

plt.tight_layout()
plt.show()
```



Zadanie 1

Wykorzustując wiedze z poprzednich zadań i powyższy przykład dodaj kilka modeli SVM posiadających różne nastawy oraz klasyfikatorów oparych o algorytm Random Forest. Jakie różnice widać pomiędzy poszczególnymi klasyfikatorami? Które klasyfikatory i z jakimi nastawami radzą sobie najlepiej?

Zadanie 2

Analizując różne sposoby rozmieszczenia danych na płaszczyźnie jakich klasyfiaktorów użyjesz do:

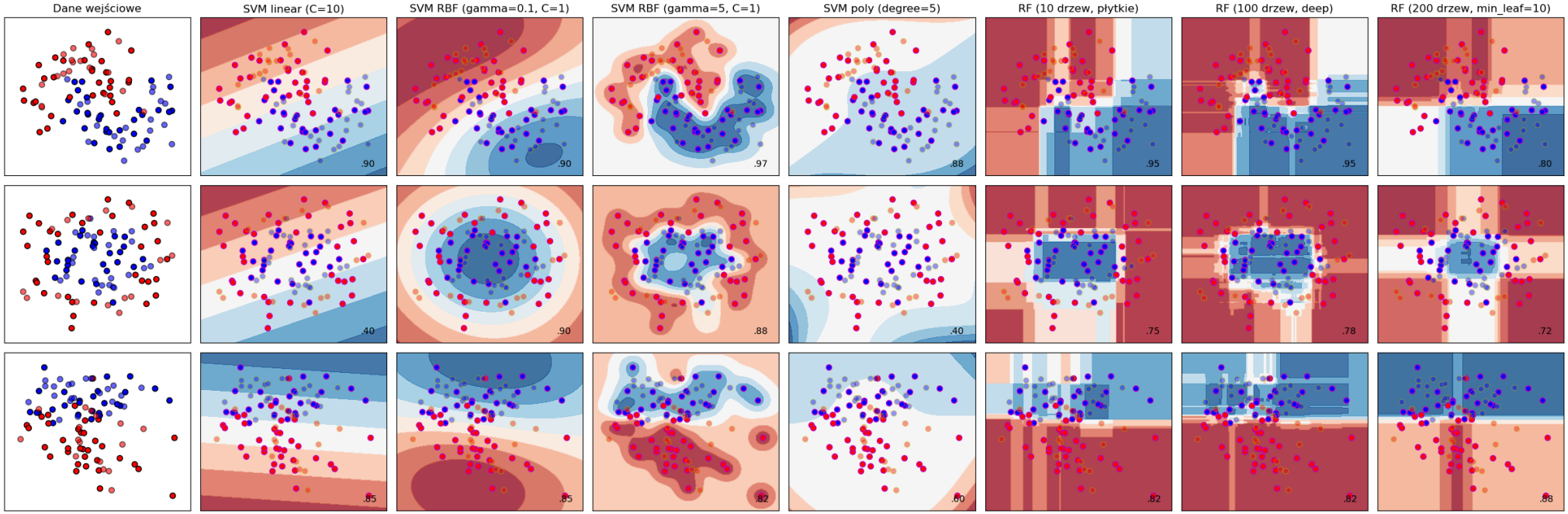
> a) Danych, gdzie klasy są od siebie dosyć wyraźnie odseparowane?

> b) Danych, gdzie nie można w prosty sposób odseparować od siebie klas?

Odpowiedź uzasadnij bazując na przebiegu laboratorium.

In [8]:
```
# Zadanie 1
names_task1 = [
    "SVM linear (C=10)",
    "SVM RBF (gamma=0.1, C=1)",
    "SVM RBF (gamma=5, C=1)",
    "SVM poly (degree=5)",
    "RF (10 drzew, płytkie)",
    "RF (100 drzew, deep)",
    "RF (200 drzew, min_leaf=10)"
]

classifiers_task1 = [
    SVC(kernel="linear", C=10),
    SVC(kernel="rbf", gamma=0.1, C=1),
    SVC(kernel="rbf", gamma=5, C=1),
    SVC(kernel="poly", degree=5, gamma='scale', C=1),
    RandomForestClassifier(max_depth=3, n_estimators=10, random_state=42),
    RandomForestClassifier(max_depth=None, n_estimators=100, random_state=42),
    RandomForestClassifier(max_depth=None, n_estimators=200, min_samples_leaf=10, random_state=42)
]

figure = plt.figure(figsize=(24, 8))
i = 1
h = .02

for ds_cnt, ds in enumerate(datasets):
    X, y = ds
    X = StandardScaler().fit_transform(X)
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.4, random_state=42)

    x_min, x_max = X[:, 0].min() - .5, X[:, 0].max() + .5
    y_min, y_max = X[:, 1].min() - .5, X[:, 1].max() + .5
    xx, yy = np.meshgrid(np.arange(x_min, x_max, h),
                         np.arange(y_min, y_max, h))

    cm = plt.cm.RdBu
    cm_bright = ListedColormap(['#FF0000', '#0000FF'])
```

```python
    ax = plt.subplot(len(datasets), len(classifiers_task1) + 1, i)
    if ds_cnt == 0:
        ax.set_title("Dane wejściowe")

    ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=cm_bright, edgecolors='k')
    ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=cm_bright, alpha=0.6, edgecolors='k')
    ax.set_xlim(xx.min(), xx.max())
    ax.set_ylim(yy.min(), yy.max())
    ax.set_xticks(())
    ax.set_yticks(())
    i += 1

    for name, clf in zip(names_task1, classifiers_task1):
        ax = plt.subplot(len(datasets), len(classifiers_task1) + 1, i)
        clf.fit(X_train, y_train)
        score = clf.score(X_test, y_test)

        if hasattr(clf, "decision_function"):
            Z = clf.decision_function(np.c_[xx.ravel(), yy.ravel()])
        else:
            Z = clf.predict_proba(np.c_[xx.ravel(), yy.ravel()])[:, 1]

        Z = Z.reshape(xx.shape)
        ax.contourf(xx, yy, Z, cmap=cm, alpha=.8)

        ax.scatter(X_train[:, 0], X_train[:, 1], c=y_train, cmap=cm_bright, edgecolors='m')
        ax.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=cm_bright, edgecolors='y', alpha=0.4)

        ax.set_xlim(xx.min(), xx.max())
        ax.set_ylim(yy.min(), yy.max())
        ax.set_xticks(())
        ax.set_yticks(())
        if ds_cnt == 0:
            ax.set_title(name)
        ax.text(xx.max() - .3, yy.min() + .3, ('%.2f' % score).lstrip('0'),
                size=10, horizontalalignment='right')
        i += 1

plt.tight_layout()
plt.show()
```



Komentarz / wnioski do Zadania 1

- SVM, kernel liniowy od razu radzi sobie bardzo dobrze (dokładność .95) na zbiorze gdzie dwie grupy kropek da się przeciąć jedną kreską, jednak na bardziej zawiłych połówkach księżyców i kołach jest już zupełnie bezużyteczny - linia prosta nie potrafi się wyginać :)
- Włączenie SVM z jądrem `RBF` działa zdecydowanie lepiej na nieliniowych zbiorach. Mała wartość `gamma=0.1` robi te granice łagodniejsze. Z kolei zawyżenie do `gamma=5` powoduje, że marginesy robią się bardziej agresywne. Obie wersje dobrze radzą sobie na księżycach i w okręgach
- SVM policzony wielomianem (stopień 5) nienaturalnie wygina przestrzeń decyzyjną - nie sprawdza się to za dobrze w typowych zadaniach.
- Lasy Losowe (Random Forest) - płytkie drzewa dają sobie świetnie radę. Model zdużą głębokością i liczbą drzew potrafi tworzyć nieco porwane krawędzie wokół trudnych danych. Ale wciąż sobie dobrze radzi w tego typu kształtach, uzyskujący bez większego tuningu świetną dokładność. Dodanie parametru `min_samples_leaf=10` dobrze wygładziło strefy decyzyjne dla RF poprzez narzucenie ograniczenia przeciwko overfittingowi

Zadanie 2 - Uzasadnienie i dobór klasyfikatora:

a) Dane, gdzie klasy są od siebie dosyć wyraźnie odseparowane (np. prosta linia lub łatwy rozdział): Zdecydowanie wystarczy użyć klasyfikatorów, które szukają prostej linii podziału – czyli prostego SVM z jądrem liniowym albo logistic regression. Takie modele wymagają mało zasobów względem bardzo zawiłych struktur analitycznych, więc odpalą się błyskawicznie.

b) Dane, gdzie nie można w prosty sposób odseparować klas liniowo (klasy "ukryte" i wymieszane np. układające się w okręgi): Tutaj proste kernele mocno zawodzą (bo nie da się prostą przedzielić okręgu). Najlepiej postawić na SVM z jądrem opartym o funkcję RBF – świetnie radzi on sobie z okręgami, po prostu je naturalnie obudowując granicą.

Drugim dobrze radzącym sobie narzedziem są drzewa - w szczególności Random Forest, których to algorytmy potrafią sprytnie i geometrycznie powycinać nieregularne luki decyzyjne odnajdując ukryty sens w pozornie trudnych danych bez problemu. Zawsze jednak trzeba w nich kontrolować dokładność z parametrami regularyzacji.